БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ПРИКЛАДНОЙ МАТЕМАТИКИ И ИНФОРМАТИКИ

ОТЧЕТ

по лабораторной работе №2

Тема: «Основы обработки цифровых изображений/сжатие изображений»

Выполнил:
Студент группы 2
Рыжов П. А.

Вариант №9

Минск, 2026

Содержание

1 Цель работы **2**

2 Задание (Вариант 9) **2**

3 Теоретические сведения **2**
3.1 Глобальная пороговая обработка 2
3.2 Сегментация (выделение границ и точек) 2

4 Результаты работы и анализ **3**
4.1 Сравнение методов глобальной пороговой обработки 3
4.2 Сравнение методов сегментации (Собель vs Лаплас) 4

5 Вывод **5**

A Листинг основных алгоритмов (image_processor.py) **6**

1 Цель работы

Реализация алгоритмов обработки изображений с использованием графического интерфейса. Выбор и сравнение двух методов из заданных категорий, а также подбор тестовой базы изображений для демонстрации преимуществ и недостатков каждого из методов.

2 Задание (Вариант 9)

Согласно таблице вариантов, необходимо реализовать:

- **Первый метод (по строке):** Глобальная пороговая обработка (2 метода на выбор, сравнить).
- **Второй метод (по столбцу):** Сегментация изображений: обнаружение точек, линий и перепадов яркости.

Для реализации было разработано десктопное приложение на языке Python с использованием библиотеки `PyQt6` для графического интерфейса и `OpenCV` для математической обработки матриц изображений.

3 Теоретические сведения

3.1 Глобальная пороговая обработка

Задача пороговой обработки — бинаризация изображения (разделение на фон и объекты).

1. **Ручной порог (Basic Thresholding):** Задается жесткое значение порога $T \in [0, 255]$. Если яркость пикселя $I(x, y) > T$, он становится белым (255), иначе — черным (0).
2. **Метод Отсу (Otsu's Thresholding):** Автоматически вычисляет оптимальный порог T, анализируя гистограмму изображения. Алгоритм перебирает все возможные значения порога и находит то, которое минимизирует внутриклассовую дисперсию (или максимизирует межклассовую) для фона и объекта.

3.2 Сегментация (выделение границ и точек)

1. **Оператор Собеля (Sobel):** Основан на вычислении первых производных (градиентов) изображения по осям X и Y. Хорошо выделяет толстые, непрерывные контуры и резкие перепады яркости, обладая при этом встроенным эффектом сглаживания шума.
2. **Оператор Лапласа (Laplacian):** Основан на вычислении второй производной. Ищет точки пересечения нуля. Крайне чувствителен к резким изменениям, поэтому идеально подходит для поиска изолированных точек, мелкой текстуры и тонких штрихов.

4 Результаты работы и анализ

4.1 Сравнение методов глобальной пороговой обработки

Для демонстрации работы метода Отсу было выбрано изображение с нотным станом (имеет идеальную бимодальную гистограмму: один пик — белый фон, второй — черные ноты). Метод Отсу автоматически вычислил оптимальный порог строго во впадине между пиками гистограммы, идеально отделив текст от фона без ручной настройки.

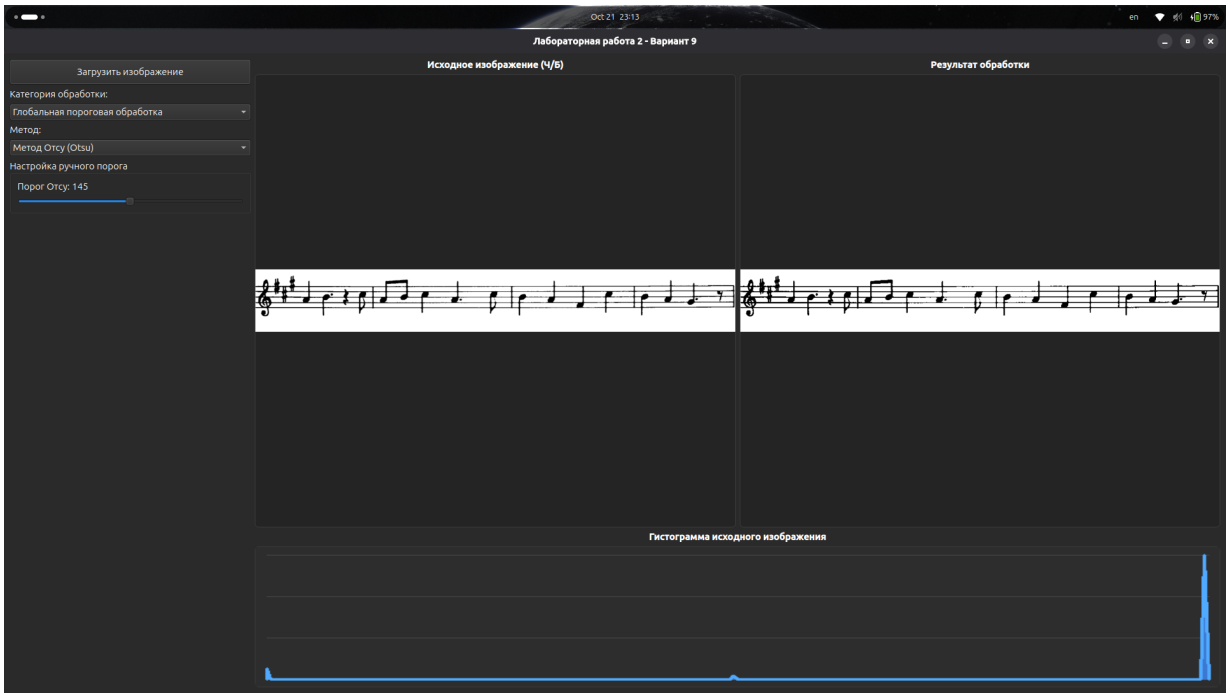


Рис. 1: Бинаризация методом Отсу (успешное применение)

Для демонстрации недостатков метода Отсу было выбрано изображение страницы «Судоку» с сильным градиентом освещения (тенью). Так как гистограмма смазана и не имеет двух четких пиков, метод Отсу терпит крах — он усредняет порог, из-за чего затененная половина листа заливается сплошным черным цветом, и данные теряются. В данном случае **ручная настройка порога** позволяет гибко подобрать значение ползунком, чтобы вытянуть данные либо из светлой, либо из темной части снимка.

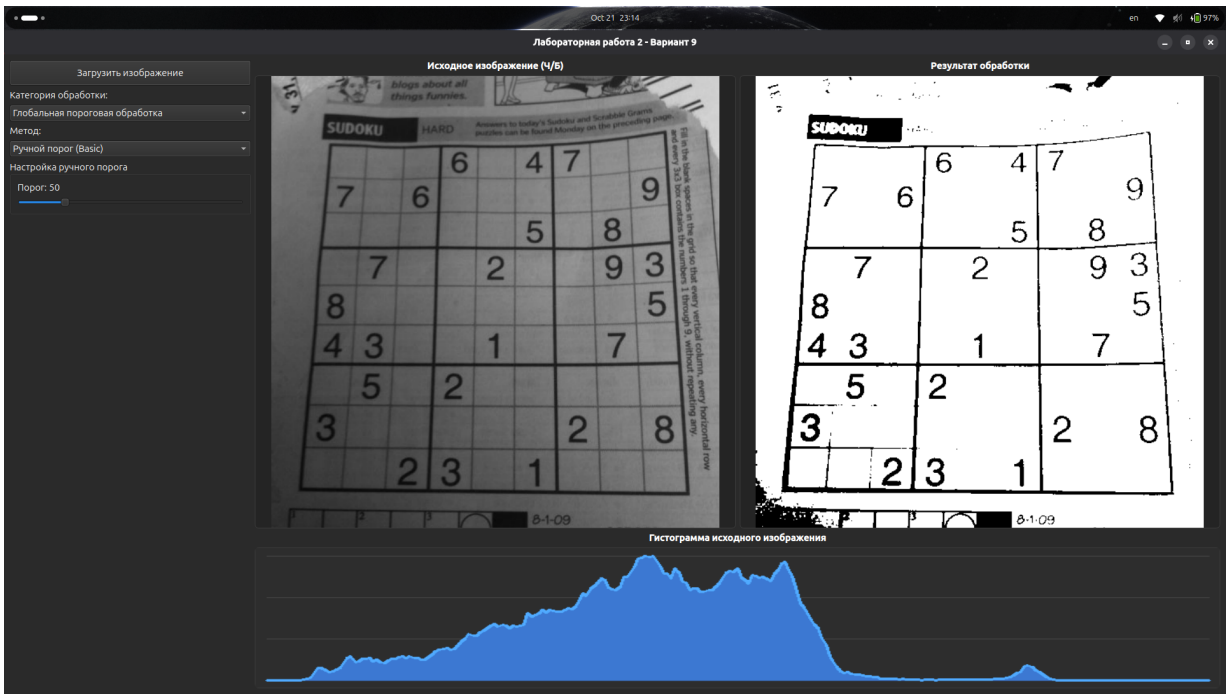


Рис. 2: Ручная пороговая обработка снимка с неравномерным освещением

4.2 Сравнение методов сегментации (Собель vs Лаплас)

Для демонстрации оператора **Собеля** было выбрано изображение фасада здания. Алгоритм, вычисляя первые производные, отлично справился с выделением непрерывных геометрических границ (оконные рамы, карнизы, перекрытия), игнорируя при этом мелкий шум благодаря маске сглаживания 3x3.

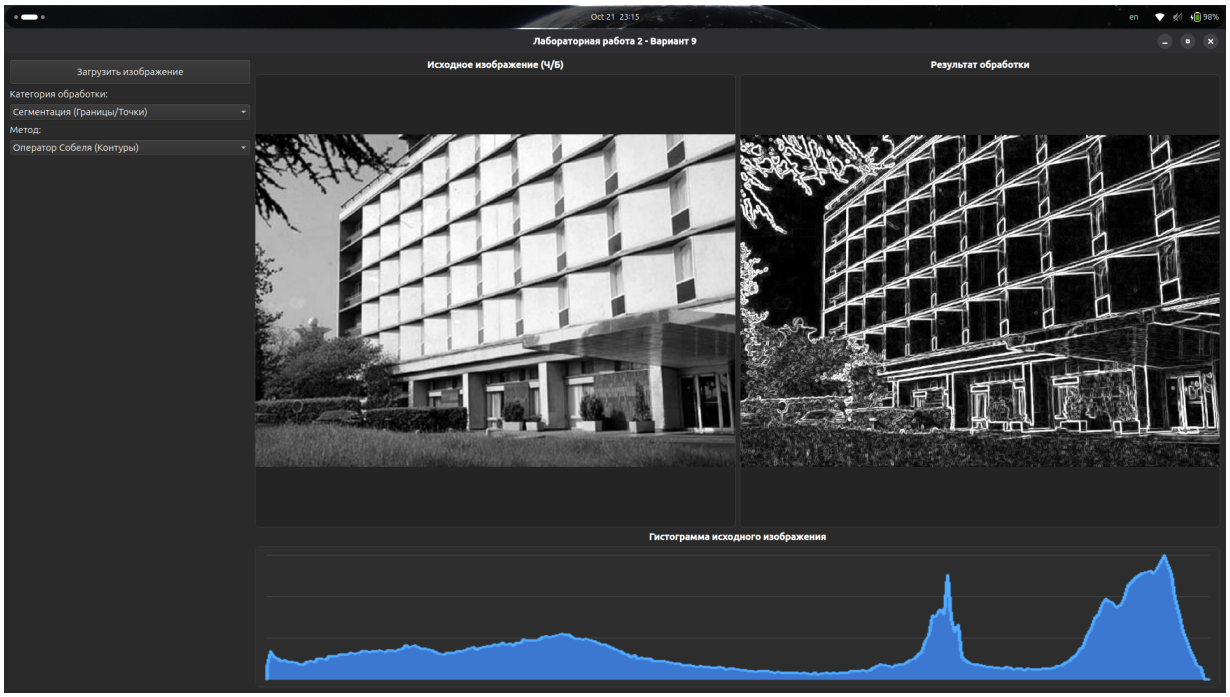


Рис. 3: Выделение контуров оператором Собеля

Для демонстрации оператора **Лапласа** была выбрана картина «Звездная ночь». Так как алгоритм использует вторые производные, он максимально чувствителен к локальным

перепадам. На результате четко видно выделение изолированных точек (звезд) и тонких линий (мазков кисти), которые оператор Собеля воспринял бы как шум и размыл.

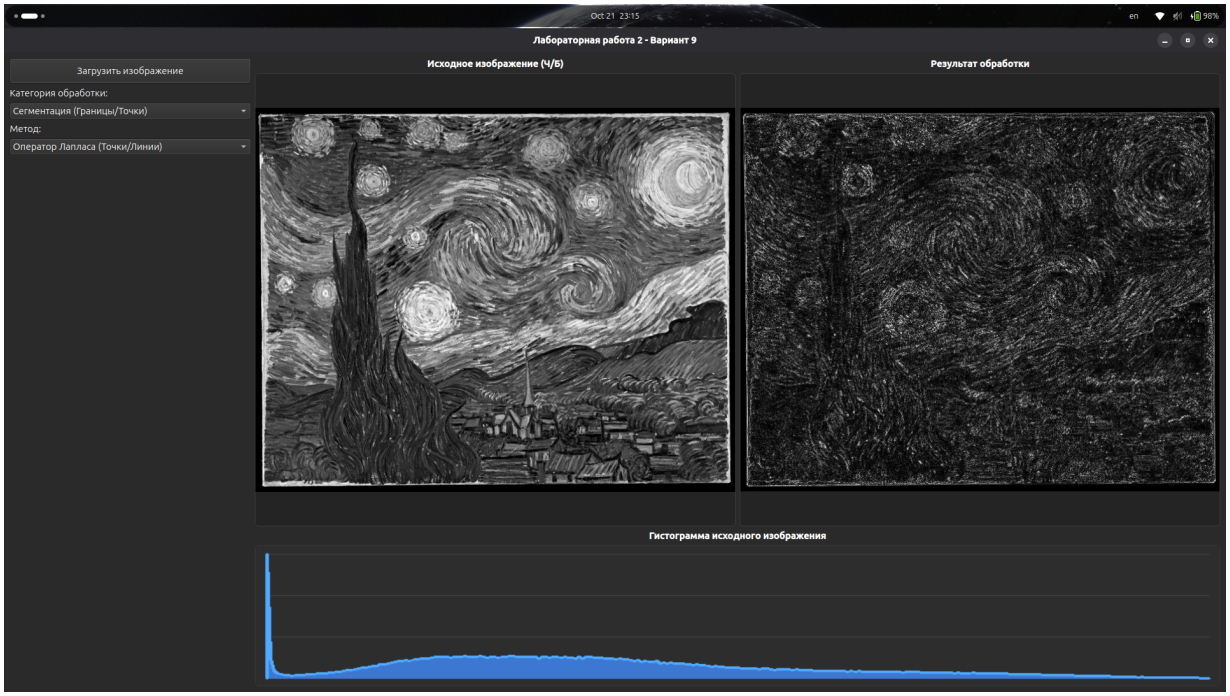


Рис. 4: Поиск точек и тонких линий оператором Лапласа

5 Вывод

В ходе лабораторной работы было разработано приложение с графическим интерфейсом для обработки изображений.

Сравнение методов показало, что автоматический **метод Отсу** является предпочтительным для документов и изображений с четко выраженным контрастом (бимодальной гистограммой). Однако при наличии градиентных теней или бликов алгоритм не справляется с задачей, и требуется применение **ручного** или локального адаптивного порога.

При пространственной сегментации выбор фильтра зависит от задачи: оператор **Собеля** лучше выделяет структурные, непрерывные контуры объектов, в то время как фильтр **Лапласа** незаменим при поиске мелких деталей, точек и текстур.

A Листинг основных алгоритмов (image_processor.py)

```python
import cv2
import numpy as np

class ImageProcessor:
    @staticmethod
    def apply_manual_threshold(image, threshold_value):
        _, result = cv2.threshold(image, threshold_value, 255, cv2.THRESH_BINARY)
        return result

    @staticmethod
    def apply_otsu_threshold(image):
        thresh_val, result = cv2.threshold(image, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
        return thresh_val, result

    @staticmethod
    def apply_sobel(image):
        sobelx = cv2.Sobel(image, cv2.CV_64F, 1, 0, ksize=3)
        sobely = cv2.Sobel(image, cv2.CV_64F, 0, 1, ksize=3)
        magnitude = cv2.magnitude(sobelx, sobely)
        return cv2.convertScaleAbs(magnitude)

    @staticmethod
    def apply_laplacian(image):
        laplacian = cv2.Laplacian(image, cv2.CV_64F)
        return cv2.convertScaleAbs(laplacian)
```